

Assignment
- 6

Q1. Write a C program to simulate the following CPU scheduling algorithms to find turnaround time and waiting time for the given problem. Assumptions. Time slice $t=2ms$ (for RR scheduling), priority assigned to the process as $P2 > P3 > P1 > P4$ (for priority scheduling)

=> #include <stdio.h>

struct process {

int pid;

int arrival;

int burst;

int priority;

int waiting;

int turnaround;

int completion;

int remaining;

};

void sort (struct process p[], int n) {

for (int i = 0 ; i < n ; i++) {

for (int j = 0 ; j < n ; j++) {

if (p[i].arrival > p[j+1].arrival)

struct process temp = p[j];

p[j] = p[j+1];

p[j+1] = temp;

}

}

}

}

```
void fcfs (struct process P[], int n) {
    sort (p, n);
    int time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival) {
            time = p[i].arrival;
            time += p[i].burst;
            p[i].completion = time;
            p[i].turnaround = p[i].completion - p[i].arrival;
            p[i].waiting = p[i].turnaround - p[i].burst;
        }
    }
}
```

```
void sjf (struct process P[], int n) {
    int done = 0, time = 0, minIndex;
    int completed[n];
    for (int i = 0; i < n; i++) completed[i] = 0;
    while (done != n) {
        minIndex = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= time && !completed[i]) {
                if (minIndex == -1 || p[i].burst < p[minIndex].burst) {
                    minIndex = i;
                }
            }
        }
        if (minIndex == -1) {
            time++;
        } else {
            time += p[minIndex].burst;
            p[minIndex].completion = time;
        }
    }
}
```

completed (min index) : t

done++

}

}

}

void rr (struct process p[], int n, int q) {

int time = 0, done = 0;

for (int i = 0; i < n; i++) {

p[i].remaining = p[i].burst;

while (done != n) {

for (int i = 0; i < n; i++) {

if (p[i].remaining > q) {

time += q;

p[i].remaining -= q;

} else {

time += p[i].remaining;

p[i].completion = time;

p[i].turnaround = p[i].completion

- p[i].arrival

p[i].remaining = 0;

done++;

}

}

}

}

void ps (struct process p[], int n) {

int done = 0, time = 0, high

```
int completed[n];  
for (int i = 0; i < n; i++) { completed[i] = 0;  
    while (done != n) {  
        for (int i = 0; i < n; i++) {  
            if (p[i].arrival <= time && !completed[i])  
                if (highindex == -1 || p[i].priority >  
                    p[highindex])  
                    highindex = i;  
        }  
        if (highindex == -1) {  
            time++;  
        } else {  
            time = p[highindex].burst +  
                p[highindex].completion = time;  
            p[highindex].turnaround = p[highindex].  
                completion  
                - p[highindex].arrival;  
            p[highindex].waiting = p[highindex].  
                turnaround  
                - p[highindex].burst;  
            completed[highindex] = 1;  
            done++;  
        }  
    }  
}
```

```
void print ( struct process p[], int n, const char method){
    printf ( "\n %s scheduler: \n", method);
    for (int i=0; i<n; i++){
        printf ( " P %d | t %d | t %d | t %d | t %d | -",
            p[i].pid, p[i].arrival, p[i].burst, p[i].
            p[i].waiting, p[i].turnaround, p[i].completion);
    }
}
```

```
int main () {
    struct process processu [4] = {
        { 1, 0, 2, 4, 1, 0, 0, 0, 0 },
        { 2, 2, 3, 4, 0, 0, 0, 0, 0 },
        { 3, 1, 3, 3, 0, 0, 0, 0, 0 },
        { 4, 3, 7, 2, 0, 0, 0, 0, 0 }
    };
    int n=4;
    struct process p1[n], p2[n], p3[n], p4[n];
    for (int i=0; i<n; i++){
        p1[i] = p2[i] = p3[i] = p4[i] = processu[i];
    }
    fcts (p1, n);
    print (p1, n, "FIFS");
    fcts (p2, n);
    print (p2, n, "SJF");
    fcts (p3, n, 2);
    print (p3, n, "Round Robin");
    fcts (p4, n);
    print (p4, n, "Priority");
    return 0;
}
```

Q2. Shortest Remaining Time First Scheduling

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX_P5
```

```
typedef struct {
```

```
    char id;
```

```
    int at, bt, rt, ct, wt, tqt;
```

```
} process;
```

```
int fss (process p[], int n, int time) {
```

```
    int min_index = -1, min_rt = INT_MAX;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (p[i].at <= time && p[i].rt > 0 &&
```

```
            p[i].rt < min_rt) {
```

```
                min_rt = p[i].rt;
```

```
                min_index = i;
```

```
        }
```

```
    }
```

```
    return min_index;
```

```
}
```

```
void sort (process p[], int n) {
```

```
    int completed = 0, time = 0;
```

```
    int total_wt = 0, total_tat = 0;
```

```
    while (completed < n) {
```

```
        int shortest = fss (p, n, time);
```

```
        if (shortest == -1) {
```

```
            time++;
```

```
            continue;
```

```
        }
```

```
        p[shortest].rt--;
```

```
        time++;
```


Q3

MLB scheduling

```
1) # include <stdio.h>
# include <string.h>
# define Max 4
typedef struct {
    char id [5];
    int at, bt, ct, wt, tat;
    char type [10];
} process;

void calc_times (process p[], int n) {
    int total-wt = 0, total-tat = 0, time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].at)
            time = p[i].at;
        p[i].ct = time + p[i].bt;
        p[i].tat = p[i].ct + p[i].wt;
        p[i].wt = p[i].tat - p[i].bt;
        time = p[i].ct;
        total-wt += p[i].wt;
        total-tat += p[i].tat;
    }

    for (int i = 0; i < n; i++) {
        p[i].id, p[i].type, p[i].at, p[i].bt,
        p[i].ct,
        p[i].tat, p[i].wt;
    }
}
```

```

void merge_sched(Procs p[], int n) {
    Procs sys_q [MAX_P], user_q [MAX_U];
    int sys_c = 0, user_c = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(p[i].type, "System") == 0)
            sys_q[sys_c++] = p[i];
        else
            user_q[user_c++] = p[i];
    }
    calc_time(user_q, user_c);
    calc_time(sys_q, sys_c);
}

```